



UNIVERSIDADE DA CORUÑA

TransitOps

Diseño y desarrollo de una aplicación
de gestión de transportes

Ciclo de vida completo del software

Pablo Manzanares López

Dirección: Paula María Castro Castro

Grado en Ingeniería Informática
Mención en Ingeniería del Software

Estructura de la presentación

01 Problema y objetivos

02 Requisitos y alcance

03 Metodología

04 Diseño y arquitectura

05 Validación y despliegue

06 Conclusiones y trabajo futuro

Problema de partida

La información operativa está repartida entre hojas, mensajes y llamadas

Caso ficticio de pyme de transporte definido mediante una entrevista simulada

Disponibilidad incierta

Un vehículo o conductor puede quedar reservado dos veces.

Historia incompleta

Reconstruir una incidencia exige buscar qué pasó y quién actuó.

Datos sin vigencia clara

Los registros obsoletos pueden volver a utilizarse por error.

Objetivos y criterios de éxito

Centralizar la operación diaria y demostrar el ciclo de vida completo del producto

Criterio	Evidencia esperada
Funcionalidad	Cuatro casos de uso completos
Calidad	Reglas verificadas en varias capas
Reproducibilidad	Ejecución y despliegue documentados
Trazabilidad	Necesidad, decisión y prueba relacionadas

Requisitos y alcance

14

requisitos
funcionales

17

reglas de
negocio

6

requisitos no
funcionales

Incluido

Catálogos y envíos
Asignación y estados
Eventos e incidencias
Usuarios e indicadores

Actores del sistema

Operador: trabajo diario
Administrador: cuentas y permisos

Fuera del TFG

Optimización de rutas
GPS y facturación
Acceso de clientes externos

Casos de uso definidos en la memoria

Caso de uso	Actor y responsabilidad	Resultado observable
CU-1 · Poner en marcha una instalación vacía	Instalador	Primer administrador creado; arranque bloqueado
CU-2 · Incorporar a una persona al sistema	Administrador: crea la cuenta Operador: realiza el primer acceso	Cuenta activa con contraseña propia
CU-3 · Ejecutar un envío de principio a fin	Operador	Envío terminal con cronología verificable
CU-4 · Retirar el acceso sin perder historial	Administrador	Cuenta desactivada; historial conservado

Metodología incremental

Incremento	Resultado integrado
S1	Esqueleto autenticado
S2–S3	Catálogos, envíos y filtros
S4–S5	Asignación, estados e historial
S6	Usuarios e indicadores
S7	Endurecimiento, sistema y despliegue

Definición de hecho

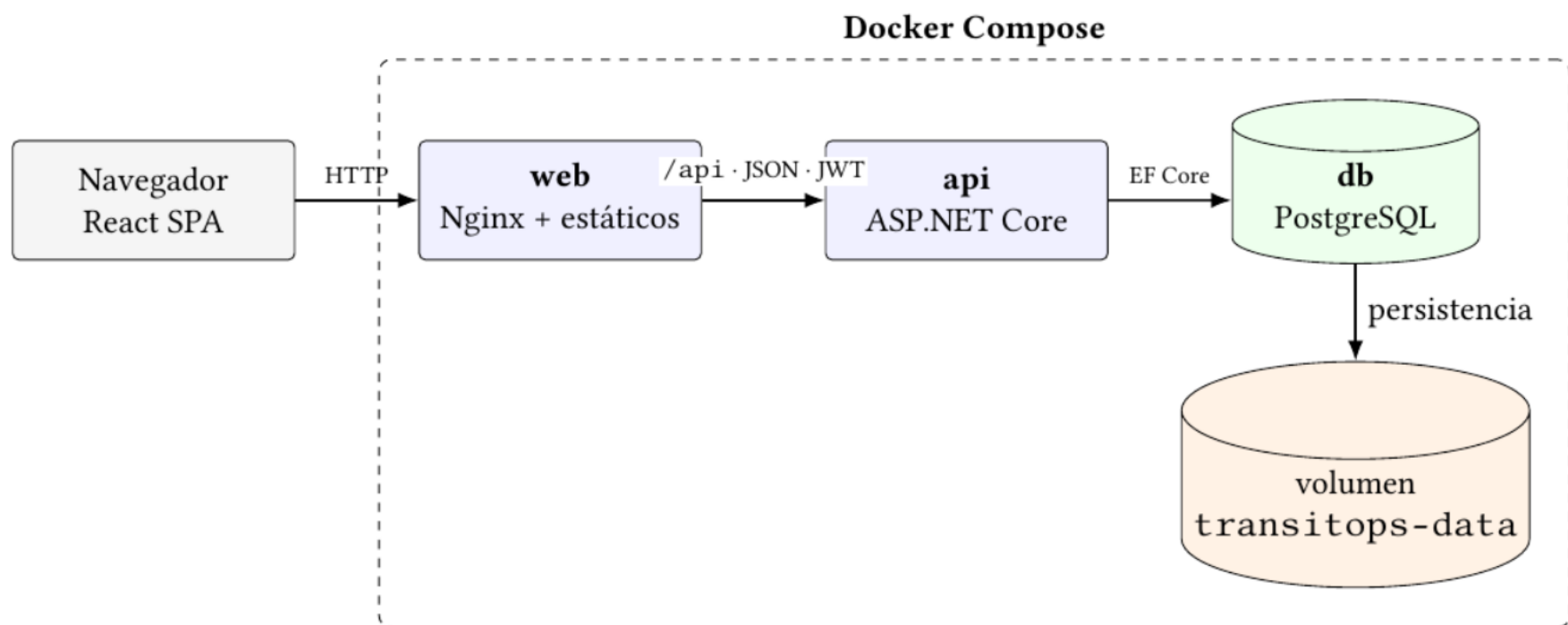
Flujo utilizable
Pruebas y compilación
Evidencia documentada

Decisión de diseño

Modelo conceptual completo
y esquema físico incremental

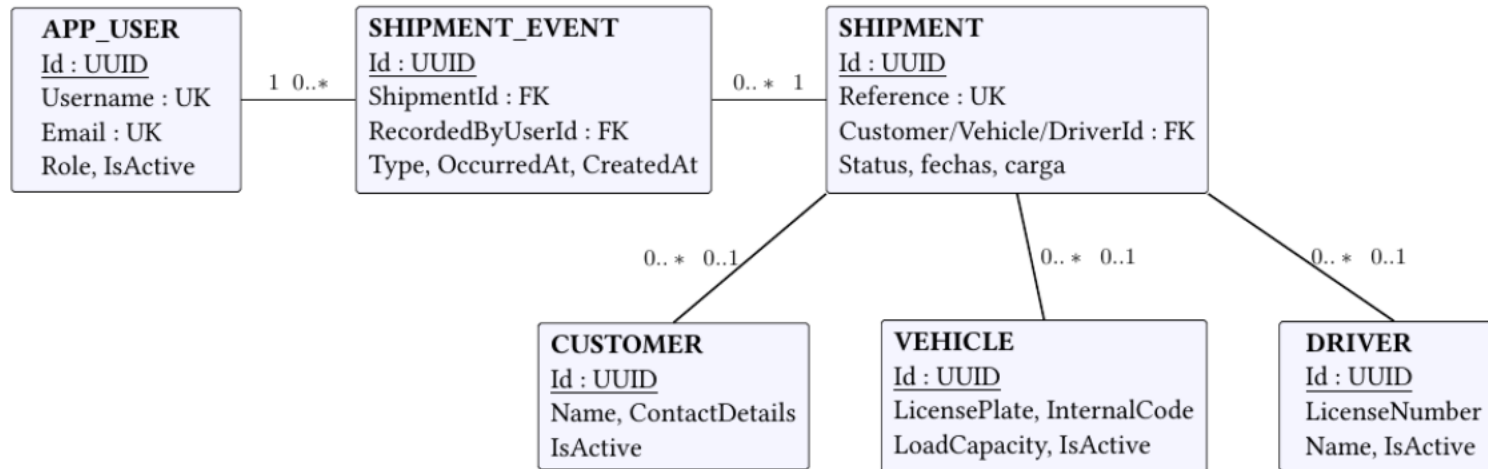
Arquitectura del sistema

React + TypeScript ASP.NET Core / .NET 10 PostgreSQL + EF Core



La API concentra autorización, validación y reglas de negocio

Modelo y reglas del dominio



Envío como núcleo

Cliente opcional y pareja de vehículo y conductor.

Historia preservada

Baja lógica en catálogos y usuarios. Eventos sin edición ni borrado.

Las invariantes críticas también se refuerzan en PostgreSQL

Estrategia de validación

139

pruebas backend
xUnit

33

pruebas frontend
Vitest + RTL

4

flujos de sistema
Playwright

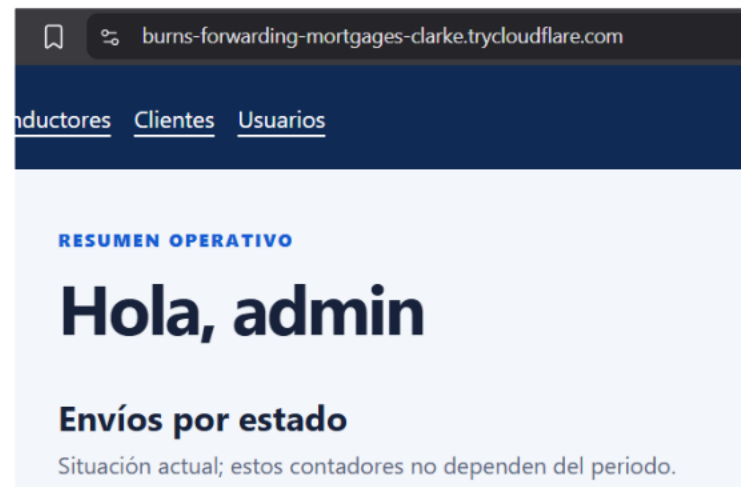
Nivel	Riesgo comprobado
Servicios y API	Reglas, contratos y autorización
PostgreSQL real	Restricciones y carreras de concurrencia
Sistema integrado	Casos de uso sobre Compose
Entorno desplegado	HTTPS, cookie y configuración efectiva

Despliegue reproducible

Etapas	Responsabilidad
GitHub Actions	Valida y publica API y web en GHCR
VM Ubuntu	Descarga imágenes y aplica Compose
Script + systemd	Comprueba salud, revisión y digests

Acceso HTTPS temporal

Cuatro servicios, sin puertos de contenedores publicados en el host



Evaluación del resultado

Criterio inicial	Evidencia final	Evaluación
Funcionalidad	RF-01 a RF-14 y cuatro casos de uso	Cumplido
Calidad	Pruebas por capas y reglas de concurrencia	Cumplido
Reproducibilidad	Compose, CI y despliegue documentado	Cumplido
Impacto empresarial	Sin evaluación con usuarios reales	No evaluado

El resultado cumple los criterios técnicos definidos al inicio sin atribuir beneficios no medidos

Evaluación basada en la trazabilidad, los resultados y las conclusiones de la memoria, pp. 61–69.

Conclusiones

Un núcleo operativo completo con un ciclo de ingeniería verificable

Las necesidades se relacionan con el diseño y las pruebas.

Cada incremento entrega una capacidad utilizable.

El sistema puede reproducirse y desplegarse siguiendo la documentación.

Gracias

Preguntas

Trazabilidad de requisitos

Requisitos	Incremento	Evidencia principal
RF-01, RF-02, RF-13	S1	Acceso, bootstrap y contrato de error
RF-05–RF-07	S2	Catálogos y conservación histórica
RF-08, RF-12	S3	Envíos, filtros y fechas
RF-09, RF-10	S4 + S7	Operación y concurrencia real
RF-11	S5	Historial, autoría y atomicidad
RF-03, RF-04, RF-14	S6 + S7	Usuarios, contraseña e indicadores

Concurrencia y seguridad

Riesgo	Garantía aplicada	Resultado
Doble reserva	Índices únicos parciales en PostgreSQL	La segunda operación recibe 409
Último administrador	Bloqueo transaccional y reevaluación	La baja incompatible recibe 409
Token accesible a JavaScript	Cookie HttpOnly, Secure y SameSite=Strict	Menor exposición del token
Permisos obsoletos	TokenVersion comprobado por petición	Revocación tras cambio sensible

Planificación y coste

Concepto	Hipótesis de cálculo	Importe
Personal	$300 \text{ h} \times 20 \text{ €/h}$	6.000,00 €
Amortización del equipo	$1.200 \text{ €} \times 2,5 / 48 \text{ meses}$	62,50 €
Licencias	Uso libre o gratuito en el proyecto	0,00 €
Infraestructura y servicios	Máquina local y cuotas gratuitas	0,00 €
Costes indirectos	Energía y conectividad estimadas	75,00 €
Total	Valoración profesional hipotética	6.137,50 €

Evolución de las pruebas

Sprint	Backend	Frontend	Sistema
S1	16	4	—
S2	29	7	—
S3	45	13	—
S4	74	19	—
S5	96	24	—
S6	127	30	—
S7	139	33	4 flujos E2E